QuickCart Black-Box API Testing Report

This report documents the test case design, execution results, and bug reports for the QuickCart REST API.

0 Contents

1 Test Case Design **2**
1.1 Header & Security Validation . . . 2
1.2 Profile & Address Management . . . 2
1.3 Product Catalog . . . 3
1.4 Cart & Checkout Logic . . . 3
1.5 Wallet & Loyalty Points . . . 4
1.6 Orders & Support Tickets . . . 5
1.7 Fuzzing & Advanced Stress . . . 5
1.8 Additional Edge-Case Validation . . . 6

2 Automated Test Execution **6**
2.1 Verification Logic . . . 7
2.2 Execution Summary . . . 7

3 Bug Reports **7**

4 Coverage Matrix **12**

5 Test File Inventory **13**

1 Test Case Design

The tables below list the named test cases; several of them expand into multiple parametrized checks in the automated suite.

1.1 Header & Security Validation

Every request must include `X-Roll-Number`. User-scoped endpoints also require `X-User-ID`.

Table 1. Header & Security Validation Test Cases

ID	Scenario [Type]	Input	Expected Output	Justification & Importance
SEC-01	Missing Roll No [M]	No `X-Roll-Number`	`401 Unauthorized`	Ensures mandatory system-wide access control.
SEC-02	Invalid Roll Type [T]	`X-Roll-Number: "ABC"`	`400 Bad Request`	Verifies data type safety for system headers.
SEC-03	Missing User ID [M]	No `X-User-ID`	`400 Bad Request`	Ensures user-scoped data cannot be accessed anonymously.
SEC-04	Negative User ID [B]	`X-User-ID: -5`	`400 Bad Request`	Verifies positive integer boundaries.
SEC-05	Auth Leak [I]	Regular user hits Admin	`403 Forbidden`	Critical for privilege escalation prevention.
SEC-06	Access Control [I]	User A hits User B ticket	`403 Forbidden`	Ensures cross-user data isolation.
SEC-07	No Headers [M]	Empty Headers	`401 Unauthorized`	Verifies default rejection of unauthenticated traffic.

1.2 Profile & Address Management

Validating constraints on user-supplied data and address invariants.

Table 2. Profile & Address Management Test Cases

ID	Scenario [Type]	Input	Expected Output	Justification & Importance
PROF-01	Min Name Length [B]	`name: "A"`	`400 Bad Request`	Ensures realistic identity data (min 2 chars).
PROF-02	Phone Length [B]	`phone: "123"`	`400 Bad Request`	Ensures exact 10-digit mobile format.
PROF-03	Phone Type [T]	`phone: 1234567890`	`400 Bad Request`	Verifies JSON string constraint for phone fields.

Continued on next page...

Table 2 (continued)

ID	Scenario [Type]	Input	Expected Output	Justification & Importance
ADDR 01	Enum Violation [I]	`label: "VACATION"`	`400 Bad Request`	Verifies adherence to allowed address types.
ADDR 02	Pincode Length [B]	`pincode: "12345"`	`400 Bad Request`	Verifies exact 6-digit postal code constraint.
ADDR 03	Pincode Format [I]	`pincode: "12A456"`	`400 Bad Request`	Verifies digit-only constraint for logistics.
ADDR 04	Default Swap [V]	Set 2nd default	`200` + Prev unset	Verifies "single default" business logic.
ADDR 05	Missing Field [M]	`{pincode: 123456}`	`400 Bad Request`	Ensures all components of an address are present.

1.3 Product Catalog

Testing visibility, search accuracy, and sorting logic for consumer-facing browse features.

Table 3. Product Catalog Test Cases

ID	Scenario [Type]	Input	Expected Output	Justification & Importance
PROD-01	State Filtering [V]	Base URL	List where `active: True`	Ensures consumers don't see unlisted items.
PROD-02	Resource Discovery [I]	ID: 9999	`404 Not Found`	Verifies graceful error handling for missing items.
PROD-03	Sort Numerics [V]	`?sort=price_asc`	JSON array, $p[i] \leq p[i+1]$	Verifies numerical sorting and API correctness.
PROD-04	Matrix Search [V]	`?cat=X&q=Y`	Combined results	Verifies multi-parameter query processing.
PROD-05	Inactive Lookup [I]	ID: [Inactive]	`404 Not Found`	Ensures single-item lookup honors state rules.
PROD-06	String Fuzzing [I]	`search: "'--"`	`200/400` (No 500)	Verifies sanitization against SQL Injection.

1.4 Cart & Checkout Logic

Verifying stateful transitions, inventory guards, and financial logic.

Table 4. Cart & Checkout Logic Test Cases

ID	Scenario [Type]	Input	Expected Output	Justification & Importance
CART-01	Min Quantity [B]	`quantity`: 0	`400 Bad Request`	Prevents nonsensical 0-item additions.
CART-02	Inventory Stock [B]	`qty`: 999999	`400 Bad Request`	Essential guard against overselling.
CART-03	State Merge [V]	Add existing	Merged quantities	Verifies correct state accumulation logic.
CART-04	Null Quantity [T]	`qty`: null	`400 Bad Request`	Verifies type-safety for cart transactions.
COUP-01	Min Threshold [B]	`SAVE10` $<$ \$500	`400 Bad Request`	Verifies minimum cart value rule enforcement.
COUP-02	Discount Caps [B]	Huge cart value	`total = max_cap`	Verifies maximum discount cap enforcement.
CHCK 01	Payment Filter [I]	`meth`: BTC	`400 Bad Request`	Ensures only white-listed payment methods.
CHCK 02	COD Threshold [B]	Value $>$ \$5000	`400 Bad Request`	Critical anti-fraud and risk management rule.
FIN-01	Tax Precision [V]	Valid Order	Total = Sub +5% GST	Ensures financial accuracy and legal compliance.

1.5 Wallet & Loyalty Points

Testing balance management and point redemption invariants.

Table 5. Wallet & Loyalty Points Test Cases

ID	Scenario [Type]	Input	Expected Output	Justification & Importance
WALL-01	Low Balance [B]	`pay`: $\$10^6$	`400 Bad Request`	Validates insufficient funds handling.
WALL-02	Sign Check [B]	`add`: $-\$10$	`400 Bad Request`	Prevents balance reduction via top-up.
WALL-03	Maximum Top-up [B]	`add`: \$100001	`400 Bad Request`	Prevents runaway balance growth.
LOY-01	Eligibility [V]	Redeem $>$ Bal	`400 Bad Request`	Ensures points are earned before spending.
LOY-02	Min Redemption [B]	`points`: 0	`400 Bad Request`	Verifies valid transaction thresholds.

1.6 Orders & Support Tickets

Testing state machine transitions, resource immutability, and cross-user support security.

Table 6. Orders & Support Tickets Test Cases

ID	Scenario [Type]	Input	Expected Output	Justification & Importance
ORD-01	Finality Guard [I]	Cancel "Delivered"	`400 Bad Request`	Verifies that settled states cannot be reversed.
ORD-02	Idempotency [I]	Double Cancel	`400 Bad Request`	Ensures system state doesn't oscillate on duplicate hits.
ORD-03	Missing Resource [I]	Invoice for ID: 999	`404 Not Found`	Verifies consistency of resource finding logic.
SUPP-01	Subject Length [B]	`subject: "Hi"`	`400 Bad Request`	Prevents low-quality/empty support tickets (min 5 chars).
SUPP-02	State Skip [I]	`OPEN -> CLOSED`	`400 Bad Request`	Verifies mandatory "In Progress" intermediate state.
SUPP-03	Immutability [I]	Update CLOSED ticket	`400 Bad Request`	Ensures records cannot be altered after resolution.
SUPP-04	Privileged Update [I]	Admin update User	`200 OK`	Verifies administrative override capabilities.
REV-01	Order-Linked [I]	Review unordered	`400 Bad Request`	Critical anti-spam/anti-fraud logic.
REV-02	Duplicate Block [I]	Double review	`400 Bad Request`	Prevents rating manipulation by single users.
REV-03	Rating Range [B]	`rating: 6`	`400 Bad Request`	Verifies standard 1–5 scale integrity.

1.7 Fuzzing & Advanced Stress

Testing system resilience against data-type violations and high-volume adversarial traffic.

Table 7. Fuzzing & Advanced Stress Test Cases

ID	Category	Examples	Expected Output	Justification & Importance
STRS-01	SQL Injection	`' OR 1=1 --, ; DROP`	`400 / 200 (Empty)`	Ensures DB security across all query parameters.
STRS-02	XSS Payloads	`<script>, onerror=`	`400 / 200 (Escaped)`	Protects other users from malicious script injection.

Continued on next page...

Table 7 (continued)

ID	Category	Examples	Expected Output	Justification & Importance
STRS-03	Type Safety	`null`, `[]`, `True`	`400 Bad Request`	Ensures the API doesn't crash on non-string inputs.
STRS-04	Large Payload	2000+ char string	`400 / 413`	Prevents memory exhaustion / DoS via string overflow.
PREC-01	Float Precision	`amount`: 0.0000001	`400 Bad Request`	Verifies minimum financial transaction thresholds.

1.8 Additional Edge-Case Validation

These extra tests were added to probe request-shape handling that was not covered by the original suite.

Table 8. Additional Edge-Case Validation Test Cases

ID	Scenario [Type]	Input	Expected Output	Justification & Importance
EDGE-01	Missing Product ID [M]	`POST /cart/add` without `product_id`	`400 Bad Request`	Confirms the API rejects incomplete cart requests.
EDGE-02	Malformed JSON [T]	Broken JSON body for `/cart/add`	`400 Bad Request`	Confirms invalid JSON is rejected safely instead of being parsed loosely.
EDGE-03	Missing Content-Type [M]	JSON body sent without `Content-Type`	`400 Bad Request` or `415`	Confirms the API enforces request format rather than accepting ambiguous payloads.
EDGE-04	Float Phone Value [T]	`phone: 123.45`	`400 Bad Request`	Confirms phone fields stay numeric-string only.
EDGE-05	Float Review Rating [T]	`rating: 4.5`	`400 Bad Request`	Confirms review ratings must be whole numbers.

2 Automated Test Execution

The automated test suite was executed using Pytest and Requests against the live API at `http://localhost:8080`.

2.1 Verification Logic

For every scenario listed in the design tables, the framework automatically validates:

1. **Status Codes:** Exact match against expected (e.g., 400 for structural, 403 for auth).
2. **JSON Structure:** Presence/format of mandatory fields (e.g., verifying `total` in cart).
3. **Data Correctness:** Numerical accuracy and business logic consistency.

2.2 Execution Summary

Category	Tests	Passed	Failed	Success Rate
Security & Headers	20	17	3	85%
Admin / Data	15	10	5	66%
Profile & Addresses	45	40	5	88%
Product Catalog	50	47	3	94%
Cart & Checkout	50	43	7	86%
Others (Fuzzing/REV)	60	54	6	90%
Additional Edge Cases	5	3	2	60%
Total	**245**	**214**	**31**	**87%**

The execution summary above reflects the last full Pytest run captured before the later requirement-matrix additions. Subsequent focused regressions found additional bugs and expanded the current test inventory.

3 Bug Reports

Each bug entry below is written in a strict request format so it can be reproduced directly from the report. The current report documents **45 unique bugs**.

Header abbreviations in the table below: **Roll** = `X-Roll-Number: 2024101088`; **UID** = `X-User-ID: 1`; **CT** = `Content-Type: application/json`.

Table 9. Bug Reports (45 Unique Bugs)

ID	Methc	Full URL	Headers	Body	Expected	Actual
BUG-01	POST	localhost:8080/api/v1/cart/add	Roll, UID, CT	{"product_id":1, "quantity":0}	400 — min qty is 1	200 OK (zero-item addition accepted)
BUG-02	GET	localhost:8080/api/v1/products/999999	Roll, CT	None	404 Not Found	400 Bad Request
BUG-03	PUT	localhost:8080/api/v1/profile	Roll, UID, CT	{"name":"NewName", "phone":"1234567890	Response includes name field	name missing from response
BUG-04	POST	localhost:8080/api/v1/addresses	Roll, UID, CT	{"label":"HOME", ...}	Response includes address_id	address_id missing
BUG-05	PUT	localhost:8080/api/v1/support/tickets/\{id\}	Roll, UID, CT	{"status":"CLOSED"} (from OPEN)	400 — must pass IN_PROGRESS first	200 OK
BUG-06	GET	localhost:8080/api/v1/products?sort=price_asc	Roll, CT (UID omitted)	None	Public access; UID optional on base route	Requires UID only when sorting is applied
BUG-07	GET	localhost:8080/api/v1/admin/products	Roll, CT	None	Products include stock field	stock field missing
BUG-08	GET	localhost:8080/api/v1/orders/\{id\}/invoice	Roll, UID, CT	None	Invoice includes gst amount	gst field missing
BUG-09	POST	localhost:8080/api/v1/cart/add	Roll, UID, CT	{"product_id":1, "quantity":1}	Full cart state (items, total)	Only {"message":"Success"} returned
BUG-10	GET	localhost:8080/api/v1/admin/coupons	Roll, CT	None	Coupon list includes code field	code field missing
BUG-11	POST	localhost:8080/api/v1/addresses	Roll, UID, CT	{"label":"HOME",... "pincode":"123A56"}	400 — digits only	200 OK (alphanumeric accepted)

Continued on next page...

Table 9 (continued)

ID	Methc	Full URL	Headers	Body	Expected	Actual
BUG-12	PUT	localhost:8080/api/v1/support/tickets/\{id\}	Roll, UID, CT	{"status":"OPEN"} on CLOSED ticket	400 — closed tickets are immutable	200 OK (ticket reopened)
BUG-13	PUT	localhost:8080/api/v1/addresses/\{id\}	Roll, UID, CT	{"pincode":"ABCDEF"	400 Bad Request	200 OK (no validation)
BUG-14	POST	localhost:8080/api/v1/orders/\{id\}/cancel	Roll, UID, CT	None	400 if already CANCELLED	200 OK (redundant cancel allowed)
BUG-15	PUT	localhost:8080/api/v1/support/tickets/\{id\}	Roll, UID, CT	User A updates User B's ticket	403 Forbidden	200 OK (cross-user update allowed)
BUG-16	GET	localhost:8080/api/v1/reviews/average?product_id=1	Roll, UID, CT	None	200 OK with JSON average	500 Internal Server Error
BUG-17	GET	localhost:8080/api/v1/wallet/balance	Roll only (UID missing)	None	400 Bad Request	200 OK (balance leaked anonymously)
BUG-18	GET	localhost:8080/api/v1/orders/\{id\}/invoice	Roll, UID, CT	None	total = products + GST	total returned as 0
BUG-19	GET	localhost:8080/api/v1/orders/\{id\}/invoice	Roll, UID, CT	None	gst = 5% of subtotal	gst returned as 0
BUG-20	POST	localhost:8080/api/v1/cart/add	Roll, UID, CT	{"product_id":1, "quantity":null}	400 — strict integer required	200 OK (null accepted)
BUG-21	GET	localhost:8080/api/v1/products/\{inactive_id\}	Roll, CT	None	404 Not Found	200 OK (internal details exposed)
BUG-22	POST	localhost:8080/api/v1/reviews	Roll, UID, CT	{"product_id":1, "rating":5, "comment":"Nice!"}	400 — product not ordered	200 OK (fraudulent review allowed)
BUG-23	POST	localhost:8080/api/v1/reviews	Roll, UID, CT	Same review by User A for Product 1 again	400 — duplicate review	200 OK (spam reviews allowed)

Continued on next page...

Table 9 (continued)

ID	Methc	Full URL	Headers	Body	Expected	Actual
BUG-24	POST	localhost:8080/api/v1/admin/coupons	Roll, CT	{"code":"FREE",..., "discount_value":11	400 — >100% not allowed	200 OK (>100% accepted)
BUG-25	POST	localhost:8080/api/v1/reviews	Roll, UID, CT	{"rating":"-",...}	400 Bad Request	404 Not Found (parser confused)
BUG-26	POST	localhost:8080/api/v1/coupon/apply	Roll, UID, CT	Apply one-time coupon, then cancel order	Coupon restored for reuse	400 (permanently consumed)
BUG-27	PUT	localhost:8080/api/v1/admin/products/\{id\}	Roll, CT	{"price":-100}	400 Bad Request	200 OK
BUG-28	PUT	localhost:8080/api/v1/admin/products/\{id\}	Roll, CT	{"stock":-50}	400 Bad Request	200 OK
BUG-29	GET	localhost:8080/api/v1/admin/users	Roll, UID, CT	None	403 Forbidden (or 401)	200 OK (admin data leaked)
BUG-30	POST	localhost:8080/api/v1/cart/add	Roll, UID, CT	{"quantity":1}	400 — product_id missing	404 Not Found
BUG-31	POST	localhost:8080/api/v1/cart/add	Roll, UID (no CT)	{"product_id":1, "quantity":1}	400 or 415 — CT required	200 OK (no CT accepted)
BUG-32	POST	localhost:8080/api/v1/orders/\{id\}/cancel	Roll, UID, CT	Cancel order after stock reduced	Stock restored on cancellation	Stock remains reduced
BUG-33	POST	localhost:8080/api/v1/wallet/add	Roll, UID, CT	{"amount":0.0001} / 123 / 12.5	400 Bad Request	200 OK (fuzzed amounts accepted)
BUG-34	POST	localhost:8080/api/v1/orders/\{id\}/cancel	Roll, UID, CT	Cancel a DELIVERED order	400 Bad Request	404 Not Found (delivered = missing)
BUG-35	PUT	localhost:8080/api/v1/addresses/\{id\}	Roll, UID, CT	{"city":"New City"} / {"label":"OFFICE"}	400 — forbidden fields	200 OK (fields silently ignored)

Continued on next page...

Table 9 (continued)

ID	Methc	Full URL	Headers	Body	Expected	Actual
BUG-36	POST	localhost:8080/api/v1/coupon/remove	Roll, UID, CT	{"code":"INVALID_CO	400 Bad Request	200 OK (invalid code removed)
BUG-37	POST	localhost:8080/api/v1/cart/update	Roll, UID, CT	{"product_id":1, "quantity":1000000}	400 — exceeds stock	200 OK (excessive qty accepted)
BUG-38	POST	localhost:8080/api/v1/cart/update	Roll, UID, CT	{"product_id":99999 "quantity":1}	404 Not Found	200 OK (nonexistent product accepted)
BUG-39	GET	localhost:8080/api/v1/orders/\{id\}/invoice	Roll, UID, CT	Read invoice after checkout	subtotal + GST = total; matches checkout total	Invoice total_amount differs from checkout
BUG-40	GET	localhost:8080/api/v1/cart	Roll, UID, CT	Read cart after adding item	total = sum of subtotals	total returned as 0
BUG-41	POST	localhost:8080/api/v1/addresses	Roll, UID, CT	Add address with is_default:true after clearing defaults	Exactly one default address	Multiple addresses remain default
BUG-42	POST	localhost:8080/api/v1/cart/add	Roll, UID, CT	{"product_id":1, "quantity":2}	subtotal = qty × unit price	subtotal returned as -16 (expected 240)
BUG-43	GET	localhost:8080/api/v1/products/\{id\}/reviews	Roll, UID, CT	Ratings 5 and 4 posted	average_rating = 4.5	average_rating returned as 4
BUG-44	PUT	localhost:8080/api/v1/support/tickets/\{id\}	Roll, UID, CT	Reopen ticket after CLOSED state	400 Bad Request	200 OK (closed ticket reopened)
BUG-45	POST	localhost:8080/api/v1/cart/update	Roll, UID, CT	{"quantity":1}	400 — product_id missing	200 OK (missing ID accepted)

4 Coverage Matrix

The table below maps the QuickCart specification areas to the test modules that exercise them.

Table 10. Coverage Matrix

Spec Area	Main Test Modules	Coverage Note
Headers & Security	`test_security.py`, `test_final_sec.py`, `test_deep_dive_7.py`, `test_additional_edge_cases.py`	Covers required headers, missing IDs, admin access, and anonymous access checks.
Profile	`test_profile.py`, `test_massive.py`	Covers name/phone validation, type safety, and update response checks.
Addresses	`test_addresses.py`, `test_deep_dive_4.py`, `test_deep_dive_5.py`	Covers label, pincode, invalid updates, and default-address behavior.
Products	`test_products.py`, `test_massive.py`, `test_deep_dive_8.py`	Covers active-only listing, lookups, filtering, sorting, and inactive-product visibility.
Cart	`test_cart.py`, `test_massive.py`, `test_additional_edge_cases.py`, `test_requirement_matrix.py`	Covers add/clear behavior, quantity validation, missing fields, update, and remove flows.
Coupons	`test_checkout.py`, `test_deep_dive_3.py`, `test_deep_dive_9.py`, `test_requirement_matrix.py`	Covers coupon apply rules, caps, reuse cases, and coupon removal.
Checkout	`test_checkout.py`, `test_deep_dive_5.py`, `test_deep_dive_7.py`, `test_deep_dive_9.py`	Covers payment method validation, empty-cart behavior, tax math, and checkout duplication.
Wallet & Loyalty	`test_wallet_loyalty.py`, `test_massive.py`, `test_requirement_matrix.py`	Covers top-up limits, pay limits, balance visibility, and exact balance changes.
Orders	`test_orders.py`, `test_deep_dive_3.py`, `test_deep_dive_5.py`, `test_deep_dive_7.py`, `test_deep_dive_9.py`, `test_requirement_matrix.py`	Covers order listing, invoices, cancellation, stock recovery, and missing-order handling.
Reviews	`test_reviews.py`, `test_deep_dive_6.py`, `test_deep_dive_8.py`, `test_massive.py`, `test_additional_edge_cases.py`, `test_requirement_matrix.py`	Covers rating bounds, comment bounds, averages, and review permission issues.

Continued on next page. . .

Table 10 (continued)

Spec Area	Main Test Modules	Coverage Note
Support Tickets	`test_support.py`, `test_deep_dive_4.py`, `test_deep_dive_6.py`, `test_requirement_matrix.py`	Covers subject/message validation, update transitions, and ticket creation response shape.
Admin Inspection	`test_admin.py`, `test_final_sec.py`, `test_deep_dive_7.py`, `test_requirement_matrix.py`	Covers users, orders, carts, addresses, coupons, tickets, and products endpoints.

5 Test File Inventory

The automated suite consists of 25 test modules totaling **103 high-level test functions**, with additional parametrized variations inside several files.

Test File	Test Count	Prime Focus
`test_massive.py`	8	High-volume Input Fuzzing
`test_deep_dive_1.py ... test_deep_dive_9.py`	32	Complex Logic & State Machines
`test_fuzzing.py`	4	Initial Type Safety & Guardrails
`test_security.py`	3	Header Integrity & Authentication
`test_addresses.py`	3	Label Enums & Pincode Logic
`test_cart.py`	3	Basic Addition & Removal
`test_checkout.py`	3	Payment Gateways & Thresholds
`test_orders.py`	3	Cancellation & Invoicing
`test_products.py`	3	Catalog Browsing & Sorting
`test_profile.py`	3	Name/Phone Formatting Rules
`test_reviews.py`	3	Feedback Integrity & Rating Math
`test_wallet_loyalty.py`	3	Insufficient Funds & Redemption
`test_admin.py`	2	Administrative Catalog Control
`test_final_sec.py`	2	Privilege Escalation Probes
`test_support.py`	2	Ticket Lifecycle & Transitions
`test_additional_edge_cases.py`	5	Request-Shape Edge Cases
`test_requirement_matrix.py`	21	Spec Coverage Gaps
Total	**103**	**Exhaustive coverage**